Penguin CODEPRO

Penguin Academy

Penguin CODEPRO

# Cheat sheet Python

**Python** es un lenguaje de programación interpretado, de alto nivel. Se caracteriza por tener una sintaxis simple y legible, lo que lo hace ideal para principiantes. Es ampliamente usado en ciencia de datos, inteligencia artificial, desarrollo web, automatización y scripting.

## VARIABLES

Las variables son como cajas en la memoria donde guardamos datos.
Les damos un nombre para acceder fácilmente y su contenido puede cambiar a lo largo del programa.

**¿Cómo nombrar una variable?**

- Puede incluir letras, números y guiones bajos (_).
- Debe empezar con una letra o guión bajo, nunca con un número.
- Evitar usar palabras reservadas de Python (como print, def, if, etc).

```
edad = 25
edad = 26  # cambia su valor
```

## TIPO DE DATOS

Todos los valores en un programa tiene un tipo de valor. En Python se dividen en los siguientes.

**INT o INTEGER (Enteros):** Son aquellos números que no tienen decimales

```
edad = 28 # int
peso = 70 # int
```

**FLOAT O FLOATING (Decimales):** Son los números separados por un punto.

```
altura = 1.80 # --> float
temperatura = 36.6 # --> float
```

**STR o STRING (Cadenas de texto):** Son valores que representan texto. Se encuentran entre comilla simples o dobles.

```
nombre = 'julio' #--> str
direccion = "Sacramento 123" #--> str
```

**BOOL (Booleano):** Son valores del tipo TRUE (Verdadero) y False (Falso) que representan valores lógicos

```
tiene_permiso = True #--> bool
esta_lloviendo = False #--> bool
```

# OPERADORES ARITMÉTICOS

En Python podemos realizar operaciones matemáticas como en una calculadora.
El intérprete lee las expresiones de izquierda a derecha y respeta las reglas de precedencia matemática (orden de operaciones).

- + ⟶ Suma
- - ⟶ Resta
- * ⟶ Multiplicación
- / ⟶ División
- // ⟶ División entera (sin decimales)
- % ⟶ Resto (módulo)

```
#Ejemplo de precedencia
resultado = 2 + 3 * 4
print(resultado)   # 14  (primero la multiplicación)

resultado = (2 + 3) * 4
print(resultado)   # 20  (los paréntesis cambian el orden)
```

# OPERADORES DE RELACIÓN O COMPARACIÓN

Se usan para comparar dos valores y devuelven siempre un booleano **(True o False).**

- == ⟶ Igual a
- != ⟶ Diferente de
- > ⟶ Mayor que
- < ⟶ Menor que
- >= ⟶ Mayor o igual que
- <= ⟶ Menor o igual que

```
# Si la expresión lógica es ciera, el resultado es True
print(10 > 5)   # Resultado -> True

# Si la expresión lógica es falsa, el resultado es False
print(10 == 5)  # Resultado -> False
```

# OPERADORES DE ASIGNACIÓN

Se usan para dar valor a una variable o actualizar su valor.

- = ⟶ Asignación simple
- += ⟶ Suma y asigna
- -= ⟶ Resta y asigna
- *= ⟶ Multiplica y asigna
- /= ⟶ Divide y asigna
- //= ⟶ División entera y asigna

```
x = 5    # asignación
x += 3   # ahora x = 8
```

# OPERADORES LÓGICOS

Se usan para combinar condiciones y devolver un valor booleano (True o False).

- **and** ⟶ Devuelve **True** si **ambas condiciones son** verdaderas.
- **or** ⟶ Devuelve **True** si **al menos una condición** es verdadera.
- **not** ⟶ **Invierte** el valor lógico **(True →False, False →True).**

```
print(5 > 0 and 5< 10)    # True
print(5 > 10 or 5 == 5)   # True
print(not(5 == 5))        # False
```

# OPERADORES IDENTIDAD

Se usan para comprobar si dos variables apuntan al mismo objeto, no solo si tienen el mismo valor.

- **is** ⟶ **True** si **ambas** variables **son el mismo** objeto.
- **is not** ⟶ **True** si **son** objetos **distintos.**

```
a = [1,2,3]
b = a
c = [1,2,3]

print(a is b) # True (apuntan al mismo objeto)
print(a == c) # True (comparación de contenido)
```

# OPERADORES PERTENCIA

Sirven para verificar si un valor está contenido en una secuencia (lista, string, tupla, diccionario).

- in ⟶ Devuelve True si el valor está presente.
- not in ⟶ Devuelve True si el valor NO está presente.

```
frutas = ["manzana", "pera", "uva"]
print("pera" in frutas)        # True
print("naranja" not in frutas) # True

texto = "python"
print("py" in texto)           # True
```

# CONCATENACIÓN

La concatenación es la acción de unir dos o más strings en una sola cadena de texto.

```
nombre = 'Claudia'
apellido = "Melgarejo"
# Concateación de textos
print(nombre + apellido)
```

## INPUT()

La función input() se usa para leer datos ingresados por el usuario desde la consola. Siempre devuelve un string (str), aunque el usuario escriba un número.

```
nombre = input("Escribe tu nombre: ")
print("Hola", nombre)

# Convertir a entero el valor ingresado
edad = int(input("Ingresa tu edad: "))
print("El próximo año tendrás", edad + 1)
```

## FUNCIONES

Una función es un bloque de código reutilizable que ejecuta una tarea cuando se invoca o se la llama. Se define con la palabra clave **"def".**

```
# Declaración de una función
def saludar():
    print("Hola, bienvenido!")

saludar() # Llamada a la función
# salida: Hola, bienvenido!
```

## FUNCIONES CON PARÁMETROS Y ARGUMENTOS

Los **parámetros** son variables que se definen en la función.
Los **argumentos** son los valores reales que pasamos al llamarla.

```
# Declaramos una función con parámetro
def saludar_persona(nombre):
    print("Hola", nombre)

# Llamada a la función con argumento
saludar_persona("María")
# salida: Hola María
```

## CONDICIONALES

Las condicionales permiten que un programa tome decisiones dependiendo de si una condición es verdadera (True) o falsa (False).

**if** ⟶ Se ejecuta **solo si** la condición es **verdadera.**

```
edad = 20
# Condicional simple
if edad >= 18:
    print("Eres mayor de edad")
    # salida: Eres mayor de edad
```

**if ... else** ⟶ Permite ejecutar una alternativa si la condición es falsa.

```
edad = 15
# Condicional compuesta
if edad >= 18:
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
```

**if ... elif ... else** ⟶ Permite evaluar múltiples condiciones.

```
nota = 75
# Condicional múltiple
if nota >= 90:
    print("Excelente")
elif nota >= 70:
    print("Aprobado")
else:
    print("Reprobado")
```

**En resumen:**

- **if** ⟶ Decisión simple.
- **if/else** ⟶ Dos caminos posibles.
- **if/elif/else** ⟶ Múltiples caminos.

## CICLO WHILE

Ejecuta un bloque de código mientras la condición sea verdadera (True). Se usa cuando no sabemos cuántas repeticiones serán necesarias.

**Importante:** si la condición nunca cambia, puede generar un bucle infinito.

```
contador = 1
while contador <= 5:
    print("Iteración", contador)
    contador += 1
```

## CICLO FOR

Se usa para recorrer secuencias como listas, strings, diccionarios o rangos de números.
Se repite una vez por cada elemento en la colección, lo que lo hace muy útil cuando ya sabemos cuántas veces queremos iterar.

```
# Ciclo for con rango
for i in range(5):
    print(i)

# Ciclo for con lista
frutas = ["manzana", "pera", "uva"]
for fruta in frutas:
    print(fruta)
```

## LISTAS

Es una colección ordenada y mutable que puede almacenar múltiples valores, incluso de distintos tipos (números, strings, booleanos, otras listas).
Se definen usando corchetes [ ].

```
# Definmos las listas
frutas = ["manzana", "pera", "uva"]
numeros = [1, 2, 3, 4, 5]
mixta = [1, "hola", True, 3.14]
```

## LISTAS - MÉTODOS Y OPERACIONES

- **lista[0]** → Accede al elemento
- **lista[i] = Valor** → cambia un elemento
- **len(lista)** → longitud (cantidad de elementos)
- **.append(x)** → agrega al final
- **.insert(i, x)** → inserta en índice i
- **.extend(otra_lista)** → concatena listas
- **.remove(x)** → elimina por valor (primera coincidencia)
- **.pop(i)** → elimina y devuelve por índice (último si no pasas i)
- **.clear()** → vacía la lista
- **.index(x)** → primer índice de x
- **.count(x)** → cuántas veces aparece x
- **.sort()** → ordena in place
- **.reverse()** → invierte in place

## DICCIONARIOS

Es una colección no ordenada y mutable que almacena datos en pares **clave → valor.**
Se definen usando corchetes { }.

```
# Crear un diccionario
persona = {"nombre": "Ana", "edad": 25}
```

- **persona["nombre"]** → Accede al valor
- **persona["edad"] = 26** → Modifica el valor
- **persona["profesion"] = "Ingeniera"** → Modifica el valor
- **del persona["ciudad"]** → Elimina el elemento por la **clave.**
- **.keys( )** → devuelve todas las claves.
- **.values( )** → devuelve todos los valores.
- **.items( )** → devuelve pares (clave, valor).
- **.get(clave)** → devuelve el valor de la clave o None si no existe.
- **.pop(clave)** → elimina y devuelve un valor.
- **.update({...})** → actualiza el diccionario.

## OBJETOS

En Python, todo es un objeto: números, strings, listas.
**Un objeto es una instancia de una clase y tiene:**

- **Atributos de clase**
  - Son compartidos por todas las instancias de la clase.
  - Se definen directamente dentro de la clase (fuera de __init__).
- **Atributos de instancia**
  - Son propios de cada objeto. Se definen en el constructor __init__ .
- **Métodos**
  - Acción que puede realizar el objeto.

```python
class Penguin:
    # Atributo de clase
    especie = "Mamifero"

    # Bloque de inicialización (constructor)
    def __init__(self, nombre):
        # Atributo de instancia
        self.nombre = nombre

    # Método
    def nadar(self):
        return f"{self.nombre} está nadando"

# Crear objeto(instancia de la clase)
pinguino = Penguin("Pingu")
```

## HERENCIA

Permite que una clase (hija) reutilice los atributos y métodos de otra clase (padre). Así podemos crear nuevas clases sin volver a escribir todo el código.

```python
# Crear una subclase que hereda de Penguin
class Pingu(Penguin):
    def volar(self):  # Nuevo método
        return f"{self.nombre} no puede volar"
```

## COMENTARIO

Son líneas de texto en el código que no se ejecutan. Sirven para explicar la lógica, documentar pasos o dejar notas para otros programadores (¡o para ti en el futuro!).

**Tipos de comentarios**

- **Comentario de una sola línea**
  - Se escriben con el símbolo #.
- **Comentario multilínea**
  - Se escriben usando """ ... """ o ''' ... '''.

```python
# Esto es un comentario de una sola línea
print("Hola")
```

```python
"""
Este es un comentario
que ocupa varias líneas
para explicar algo más largo
"""
```

## CHALLENGE

Es un reto o ejercicio práctico diseñado para poner a prueba los conceptos aprendidos. Su objetivo es que el estudiante aplique la teoría en un problema real o simulado, desarrollando lógica y resolviendo situaciones usando código.

Normalmente un **challenge** combina varios temas (variables, condicionales, bucles, funciones, etc.) y busca fortalecer el pensamiento computacional a través de la práctica.

## PENSAMIENTO COMPUTACIONAL

Es una forma de resolver problemas inspirada en la lógica de la programación. Consiste en descomponer un problema complejo en pasos más simples y manejables, identificar patrones y diseñar soluciones sistemáticas que una computadora pueda ejecutar.

# PILARES PRINCIPALES DEL PENSAMIENTO COMPUTACIONAL

- **Descomposición →** dividir el problema en partes más pequeñas.
  - **Ejemplo:** "hacer un sándwich" en pasos: cortar pan, poner relleno, cerrar pan.
- **Reconocimiento de patrones →** encontrar similitudes en problemas o datos.
  - **Ejemplo:** notar que todas las frutas se pueden recorrer con un bucle sin importar cuáles sean.
- **Abstracción →** enfocarse en lo importante y omitir detalles irrelevantes.
  - **Ejemplo:** para calcular un promedio no importa cómo se obtuvieron los números, solo la lista final.
- **Diseño de algoritmos →** crear una secuencia ordenada de pasos para resolver el problema.
  - **Ejemplo:** escribir un pseudocódigo para ordenar una lista de números.

# PSEUDOCÓDIGO

Es una forma de escribir instrucciones de manera estructurada y clara, sin necesidad de usar la sintaxis exacta de un lenguaje de programación.

Se utiliza para planear algoritmos y expresar la lógica de un programa en pasos entendibles tanto para humanos como para programadores.

**Ejemplo de pseudocódigo (sumar dos números):**

1. Pedir número1
2. Pedir número2
3. Calcular suma = número1 + número2
4. Mostrar suma

```
# Pedir datos al usuario
numero1 = int(input("Ingresa el primer número: "))
numero2 = int(input("Ingresa el segundo número: "))

# Calcular suma
suma = numero1 + numero2

# Mostrar resultado
print("La suma es:", suma)
```

# ALGORITMO

Es un conjunto ordenado y finito de pasos que se siguen para resolver un problema o realizar una tarea.
En programación, un algoritmo se traduce en código para que la computadora lo ejecute paso a paso.

**Características de un buen algoritmo**

- **Finito →** debe terminar en algún momento.
- **Preciso →** cada paso es claro y no ambiguo.
- **Ordenado →** los pasos se ejecutan en secuencia lógica.
- **Eficiente →** resuelve el problema con el menor uso de recursos posible.

**Ejemplo algoritmo para Calcular promedio de 3 notas**

```
1. Pedir nota1, nota2 y nota3
2. Sumar las tres notas
3. Dividir la suma entre 3
4. Mostrar el resultado
```

**Ejemplo en Python**

```
# Obtenemos los números
nota1 = 15
nota2 = 18
nota3 = 12

# Calculamos el promedio
promedio = (nota1 + nota2 + nota3) / 3

# Mostramos el resultado
print("El promedio es:", promedio)
```

## DEPURACIÓN (DEBUGGING)

Proceso de encontrar y corregir errores en el código. **Curiosidad:** El término “bug” se popularizó cuando Grace Hopper encontró un insecto real en un computador en 1947.

**Pon print( )** en puntos críticos de tu programa para ver valores y flujo de ejecución y así detectar hasta dónde llega el código antes de fallar.

## NOTA CURIOSA: EL ZEN DE PYTHON

**El Zen de Python**, escrito por Tim Peters, es un conjunto de principios que expresan la filosofía y el estilo de programación en Python.

No son reglas estrictas, sino guías de diseño que ayudan a escribir código más claro, legible y “pitónico”.

**Puedes verlo directamente en Python escribiendo:**

```
import this
```

## ZEN DE PYTHON

1. Bello es mejor que feo.
2. Explícito es mejor que implícito.
3. Simple es mejor que complejo.
4. Complejo es mejor que complicado.
5. Plano es mejor que anidado.
6. Espaciado es mejor que denso.
7. La legibilidad cuenta.
8. Los casos especiales no justifican romper las reglas.
9. La practicidad gana a la pureza.
10. Los errores nunca deben pasar silenciosamente.
11. A menos que se silencien explícitamente.
12. Ante la ambigüedad, evita la tentación de adivinar.
13. Debe haber una —y preferiblemente solo una— forma obvia de hacerlo.
14. Ahora es mejor que nunca.
15. Aunque nunca suele ser mejor que ya mismo.
16. Si la implementación es difícil de explicar, es una mala idea.
17. Si la implementación es fácil de explicar, puede ser una buena idea.
18. Los espacios de nombres son una gran idea — ¡usémoslos más!